- Integrated into an API gateway (for example, ACMEHR)
- A library embedded directly into the application (for example, ACMEProject)
- A sidecar container running alongside the app in a pod (for example, ACMEFinance)

Regardless of the deployment model, the authorization client is responsible for intercepting requests, evaluating them against the policy, and enforcing the outcome—all performed locally and with minimal latency.

Carol still authors policies centrally in the Policy Administration Point (PAP). These are distributed to each PDP as part of a versioned policy management workflow.

The PDP needs more than just the policy to evaluate requests effectively; it also needs contextual data: attributes about the principal (such as Alice's role or department), the resource (for example, classification or ownership), and the environment (including time of day or network risk level). This data is retrieved from Policy Information Points (PIPs), which may include identity providers, databases, time servers, or telemetry services.

To avoid fetching this data with every request, each authorization client maintains an entity cache—a local store of attributes for principals and resources. These caches are periodically refreshed from the PIPs, allowing the PDP to make decisions quickly and independently.

This distributed approach enhances resilience, performance, and scalability. It also aligns well with modern deployment models, such as microservices, edge computing, and zero trust. The trade-off is increased operational complexity: caches must remain fresh, policy synchronization needs to be reliable, and the behavior of PDPs must stay consistent across deployments.